

NeetAspirants

An anonymous community and peer-support app for NEET exam aspirants in India. Users sign up with email + password, are assigned a random anonymous alias, and post/chat in topic-based subforums (Motivation, Study Stress, Burnout, Exam Anxiety, etc.) without revealing their real identity.

Tech stack

Backend — apps/api

- **Java 21**, **Spring Boot 3.5** (Maven)
- **Spring Web** — REST controllers
- **Spring Data JPA / Hibernate** — persistence (`ddl-auto=update`)
- **Spring Security** — JWT-based auth (via `jjwt` 0.12.6), no sessions
- **Spring WebSocket + STOMP** — real-time chat rooms, presence, typing indicators
- **MySQL** — primary datastore (`neetaspirants` schema)
- **Qdrant** — vector database for semantic post search (talked to over its REST API)
- **Groq API** (`llama-3.3-70b-versatile`) — powers an AI Assistant companion feature
- **Lombok**

Frontend — apps/web

- **Next.js 16** (App Router, JavaScript — not TypeScript)
- **React 19**
- **Tailwind CSS 4** with custom design tokens (`bg-surface` , `text-ink` , `text-muted` , `text-accent` , `text-on-accent`), dark theme by default
- **@stomp/stompjs** + **sockjs-client** — WebSocket chat client

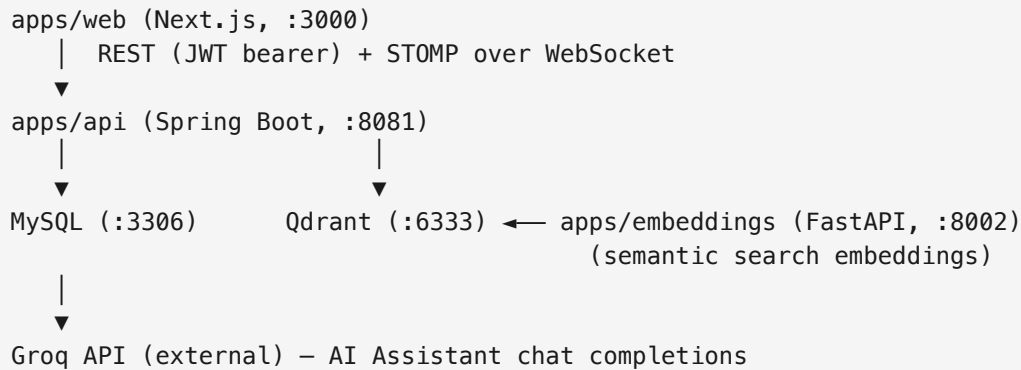
Embeddings service — apps/embeddings

- **Python / FastAPI**
- **sentence-transformers** (PyTorch backend) — generates embeddings for semantic search, feeding into Qdrant

Infra — infra

- **Qdrant** run as a local standalone binary (not Docker) for dev
- Local **MySQL** (Homebrew)
- `run-local.sh` — dev convenience script to start Qdrant and export DB creds

Architecture



Backend package layout (`com.neetaspirants.api`):

- `domain` — JPA entities: `User` , `AnonymousProfile` , `Post` , `Comment` , `Vote` , `Subforum` , `SubforumMembership` , `ChatRoom` / `ChatMessage` / `ChatRoomMember` , `Notification` , `Follow` , `SavedPost` , `DailyActivity`
- `repository` — Spring Data JPA repositories
- `service` — business logic (post/comment/vote, chat, notifications, activity, follow, subforum)
- `web` — REST controllers (auth, posts, comments, votes, subforums, chat, notifications, profile, search, saved posts, uploads, AI assistant, app-help)
- `security` — JWT filter/service, authenticated-profile resolution
- `ws` — STOMP chat controller, room presence, auth interceptor
- `config` — security config, WebSocket config, Qdrant startup wiring, dev data seeder

Key features

- Anonymous posting/commenting with randomly generated aliases per user
- Topic subforums ("communities") with membership
- Upvote/downvote on posts and comments
- Real-time chat rooms (category rooms, live presence, typing indicators, scheduled/trending/recently-active sorting) via STOMP/WebSocket
- Notifications (replies, upvotes, mentions)
- Follow system, saved posts, daily activity tracking
- Semantic post search (Qdrant + sentence-transformers embeddings)
- Post image uploads (served from local `uploads/` dir)
- Groq-backed AI Assistant chat (`/assistant`) and a separate public "App Help" widget

AI capabilities

The app has two distinct LLM-backed features (both served by the Java API, which calls out to Groq) plus a semantic search pipeline built on local embeddings and a vector database — three separate AI subsystems in total.

1. AI Assistant — supportive companion chat (/assistant)

- Backed by Groq's `llama-3.3-70b-versatile` model (`AssistantService` + `GroqClient`)
- System-prompted as a warm, practical companion for NEET exam stress and burnout — study tips, encouragement, a listening ear
- Explicitly scoped away from being a therapist: it will not diagnose, and is instructed to gently redirect any user expressing self-harm intent or a mental health crisis toward a trusted adult, professional, or crisis helpline

2. App Help widget — product Q&A chat

- Also Groq-backed, but with a separate `AppHelpService` and its own system prompt scoped strictly to "how do I use this app"
- Given a structured description of every app surface (Home feed, Communities, posting, voting/saving/commenting, Rooms, Notifications, Profile) so it can answer navigation questions accurately
- Deliberately redirects emotional-support or exam-content questions to the AI Assistant instead of answering them itself — keeps the two assistants' concerns separated

3. Semantic search — embeddings + vector similarity

- **Indexing:** on every post creation, `PostService` sends the post's title + body to the `apps/embeddings` FastAPI service, which encodes it with the `sentence-transformers` model `all-MiniLM-L6-v2` into a normalized vector, then upserts that vector into Qdrant's `posts` collection (best-effort — a post still saves even if the embedding/Qdrant step fails, since they run as separate local processes)
- **Querying:** `SearchService` embeds the user's search string the same way, asks Qdrant for the nearest post vectors, then hydrates those hits back into full post summaries from MySQL — so search matches on meaning rather than exact keywords

Both chat features share one Groq HTTP client and differ only in system prompt, so the two "personalities" (companion vs. product-help) stay behaviorally distinct while reusing the same integration code.

Running locally

Requires MySQL running locally with a `neetaspirants` database and a `neetaspirants_app` user (credentials in `infra/.mysql_app_credentials.local`, not committed).

```
# 1. Qdrant (vector DB)
cd infra
QDRANT__STORAGE__STORAGE_PATH="$(pwd)/qdrant/storage" ./qdrant/bin/qdrant

# 2. Embeddings service (optional, needed for semantic search)
cd apps/embeddings && source .venv/bin/activate && uvicorn main:app --port 8002
```

```
# 3. API (needs DB_USER / DB_PASSWORD env vars)
cd apps/api
DB_USER=neetaspirants_app DB_PASSWORD=<password> ./mvnw spring-boot:run

# 4. Web
cd apps/web && npm run dev
```

Ports: web 3000 , api 8081 , embeddings 8002 , qdrant 6333 (+ 6334 gRPC), MySQL 3306 .

The API also needs a `GR00_API_KEY` env var set for the AI Assistant / App Help features to work; without it the rest of the app still functions.